

Design Document: C-Subset Compiler for the AArch64 Instruction Set (on a Raspberry Pi 4B)

Start Date: 03/20/2026

End Date: 05/10/2026

Author: Mateo McKee

Professor: Najem Ziad

Purpose: Compiler Construction (CS4713) semester project

Status: Complete

1. Project Summary

This project is my submission for the CS4713 Compiler Construction semester project. The goal is to create a fully functional source-to-executable compiler that translates a small subset of the C language into AArch64 assembly (without preprocessing).

The goal is to touch on nearly every aspect of compiler design discussed in the course. This includes lexical analysis, parsing, semantic analysis, intermediate representation design, and code generation.

My compiler is called **nacc** (not a c compiler), and the GitHub repository can be found here:

<https://github.com/mateomckee/nacc>

Objectives: Attain hands-on experience writing compilers, learn compiler and software design principles, engineer complex problems into working solutions, learn more about assembly and the AArch64 instruction set architecture, create a working source-to-executable compiler.

Thank you for taking the time to read about my project. After what must have been around 100 hours of programming, 3,000 lines of code, and a semester coursework, I'm proud to have finally built my own. Completely from scratch, entirely written in C with no dependencies, frameworks, AI code, or even an IDE.

2. Project Timeline

- Friday 03/20/2026: Team formation and project idea sent to instructor
- Sunday 03/22/2026: Instructor amends/approves all project ideas
- Sunday 03/29/2026: Any remaining project ideas are resolved and approved
- Sunday 04/19/2026: Due date for part-I of the project
- Sunday 05/03/2026: Due date for part-II + report
- Week of 05/04/2026: Selective team interviews

3. Project Scope

Building a source-to-executable compiler from scratch is a large engineering problem, and requires a lot of time and dedication to execute properly. For the given scope and timeframe of

this course project, I have decided to design nacc around a small subset of the C language, targeting only the AArch64 assembly language.

Every component is written by hand. The lexer, parser, semantic analysis, intermediate representation, and code generation. These are described in detail in Section 5.

3.1 C-Subset Definition

Will include:

- Types: int, char, int*, char*
- Arrays: int, char
- Control Flow: if/else, while, for
- Functions: declaration, definition, recursion
- Expressions: arithmetic, comparison, logical operators, assignment, compound assignment (+, -, *, /), increment/decrement (++/--)
- Variables: local and global
- Pointers: address-of (&), dereference (*), dereference assignment (*ptr = val)
- I/O: printf/scanf via libc linking
- 1 main function with no parameters

Will **NOT** include:

- Enums, structs, unions
- switch, do-while
- Preprocessor (#define, #include)
- Floating point (double, float)
- Function pointers, ternary operators
- Anything not explicitly stated is excluded

Known limitations:

- **No optimization:** code generation is done through naive stack spilling, no register allocation or liveness analysis (I want to work on this soon as a new project)
- **No array bounds checking:** out of bounds access is undefined behavior
- **Single file only:** no multi-file compilation or linking
- **First error only:** compilation stops at the first error encountered
- **AArch64 Linux only:** output targets Linux ELF AArch64

4. Dependencies

GCC: Assembles and links the output .s file into a native binary (gcc output.s -o program)

Make: Builds nacc itself with a single command

5. Code & Compiler Design

nacc is written entirely in C and split into six programs, each covering a different stage of compilation. The full pipeline is:

```
source.c -> lexer -> parser -> sema -> tac_gen -> codegen -> output.s ->
gcc -> binary
```

nacc is an ahead-of-time (AOT) compiler. Source code is fully compiled to native AArch64 assembly before execution. The assembling and linking step is delegated to GCC, which handles libc integration automatically.

Overview: Implementation and AI Use

All components were written by hand. I wanted to take on the challenge of building a compiler from scratch instead of using tools like ANTLR to simplify the process.

I used AI as a brainstorming and design tool, and to help put into practical use the theory I learned from the course textbook, Engineering a Compiler, 3rd Edition by Keith D. Cooper and Linda Torczon. I did not use AI to implement my compiler.

I also used this project as a chance to learn nvim and git CLI workflows instead of relying on VS Code. The entire project was built in a terminal.

Project Structure

```
nacc/
├── src/
│   ├── main.c           # entry point, drives the pipeline
│   ├── lexer.h/.c       # hand-written scanner, produces token stream
│   ├── parser.h/.c      # LL(1) recursive descent parser, produces AST
│   ├── sema.h/.c        # semantic analysis, scope stack, type checking
│   ├── tac.h/.c         # three-address code IR generation
│   ├── codegen_aarch64.h/.c # AArch64 assembly emission
│   └── util.h/.c        # error reporting, memory helpers
├── tests/
└── Makefile
```

5.1 lexer.c / lexer.h

Status	Complete
Input	char* source (contents of input file)
Output	Token stream (consumed on demand by parser)

The lexer performs lexical analysis on the source file. It reads characters one at a time and groups them into tokens, which are the smallest meaningful units of the language. There are no regular expressions or automata involved. The lexer is a hand-written scanner implemented as a large switch statement over the current character.

Key design decisions:

- Hand-written scanner: no regex engine, no NFA/DFA construction. The scanner loop directly implements what a DFA would do, reading one character at a time.
- On-demand tokenization: the lexer does not produce a full token array upfront. The parser calls `next_token()` each time it needs the next token, keeping only two tokens in memory at once (current and previous).
- Token struct: stores a pointer and length into the original source buffer (no heap allocation per token), plus the line number for error reporting.

- Keywords: scanned as identifiers first, then matched against a keyword table using `strncmp` before returning.
- Comment support: single-line (`//`) and multiline (`/* */`) comments are skipped recursively by calling `next_token()` after consuming the comment.

The lexer defines the following token kinds:

- Literals: `TOK_INT_LIT`, `TOK_CHAR_LIT`, `TOK_STRING_LIT`
- Identifiers: `TOK_ID`
- Operators: `TOK_PLUS`, `TOK_MINUS`, `TOK_STAR`, `TOK_SLASH`, `TOK_EQUAL`, `TOK_PLUSPLUS`, `TOK_MINUSMINUS`, `TOK_PLUSEQ`, `TOK_MINUSEQ`, `TOK_MULTEQ`, `TOK_DIVEQ`, `TOK_AMPERSAND`, `TOK_NOT`, and comparison/logical operators
- Keywords: `TOK_INT`, `TOK_CHAR`, `TOK_VOID`, `TOK_IF`, `TOK_ELSE`, `TOK_WHILE`, `TOK_FOR`, `TOK_RETURN`
- Delimiters: `TOK_LPAREN`, `TOK_RPAREN`, `TOK_LBRACE`, `TOK_RBRACE`, `TOK_COMMA`, `TOK_SEMICOLON`
- Special: `TOK_EOF`, `TOK_ERROR`

5.2 parser.c / parser.h

Status	Complete
Input	Token stream (via <code>Lexer*</code>)
Output	<code>ASTNode* root</code> (root of the abstract syntax tree)

The parser performs syntactic analysis. It consumes tokens from the lexer and builds an Abstract Syntax Tree (AST) representing the structure of the source program. It directly skips the parse tree (concrete syntax tree) and builds the AST in one pass, which is standard practice in production compilers.

Key design decisions:

- LL(1) recursive descent: hand-written, top-down, one token of lookahead. Each production rule in the grammar maps directly to a C function.
- Grammar: the full LL(1) grammar is documented as a comment block at the top of `parser.c`, covering all productions from program down to primary expressions.
- Braceless statements: `if/else`, `while`, and `for` support both braced blocks and single-statement bodies (`if (x) return x;` is valid).
- AST node: a single generic `ASTNode` struct with `NodeKind` kind, `TypeKind` type, `Token` token, and four child pointers (`left`, `right`, `extra`, `extra2`, `next`). The `next` pointer is used for sibling chaining in linked lists (function list, statement list, param list, arg list).
- Expression precedence: handled structurally through a chain of parse functions, from `parse_assign` down to `parse_primary`. Each level calls the next, naturally encoding precedence without explicit tables.
- Postfix vs prefix: postfix `++/--` is handled in `parse_primary` after parsing an identifier, while prefix `++/--` is handled in `parse_unary`.
- Global vs function: `parse_program` peeks past the type and identifier to determine whether a top-level declaration is a function (followed by `'('`) or a global variable. Functions are stored in `node->left`, globals in `node->right`.

The node convention used throughout the compiler:

- `NODE_FUNC`: left=params, right=body
- `NODE_IF`: left=cond, right=then, extra=else
- `NODE_WHILE`: left=cond, right=body
- `NODE_FOR`: left=init, right=cond, extra=body, extra2=post
- `NODE_DECL`: left=initializer, token=variable name
- `NODE_RETURN`: left=expr
- `NODE_BINOP`: left=left operand, right=right operand, token=operator
- `NODE_UNOP`: left=operand, token=operator
- `NODE_CALL`: left=args (linked via next), token=function name

5.3 sema.c / sema.h

Status	Complete
Input	ASTNode* root (parser output)
Output	Annotated AST (each expression node has TypeKind type set)

Semantic analysis validates the AST and annotates every expression node with its resolved type. It is implemented as a recursive DFS walk over the AST, similar in structure to the TAC and codegen passes that follow.

Key design decisions:

- Two-pass architecture: the first pass (`collect_functions`) scans the function list and registers all function names, return types, and parameter types into a global function table. This allows functions to call each other regardless of declaration order. The second pass (`sema_node`) does the full walk.
- Scope stack: implemented as a linked list of Scope structs. Each Scope holds a fixed array of Symbol entries. Entering a block pushes a new scope; leaving pops it. Symbol lookup walks from the innermost scope up to the global scope.
- Global scope: initialized at startup and never popped. Global variables are declared into it by checking `depth == 0`.
- Type annotation: every expression node gets its `TypeKind` set during the walk. Literals are annotated directly (`INT_LIT -> TYPE_INT`), identifiers are looked up in the scope stack, and operators use rules to determine result type.
- Function calls: looked up in the global function table. Argument count and types are verified against the function signature. `printf` and `scanf` are pre-registered as variadic (`param_count = -1`) and skip type checking.
- Type compatibility: `int` and `char` are considered compatible (C treats `char` as a small `int`). Pointer and non-pointer types are incompatible. Violations produce a compile-time error.

5.4 tac.c / tac.h

Status	Complete
Input	ASTNode* root (sema-annotated AST)
Output	TACGen* containing a flat dynamic array of TACInstr

The TAC generation pass walks the annotated AST and emits a flat list of three-address code (TAC) instructions. TAC is the intermediate representation (IR) used by nacc. Each instruction has at most one operator and three named operands (result, op1, op2), making it straightforward to translate to assembly.

Key design decisions:

- Quadruple format: each TACInstr stores (kind, result, op1, op2) as fixed-size char arrays. Operands are strings: variable names, temp names (t0, t1, ...), label names (L0, L1, ...), integer literals, or string literal labels (str0, str1, ...).
- Two tac functions: tac_node returns char* for expression nodes (the operand name holding the result) and returns NULL for statement nodes. This mirrors the distinction between expressions (produce a value) and statements (do not).
- Argument ordering fix: when a function call appears as an argument to another call, all arguments are fully evaluated before any TAC_ARG instructions are emitted. This prevents inner call results from being clobbered before the outer call is made.
- String literals: emitted inline as TAC_GLOBAL instructions wherever they appear. Codegen collects all TAC_GLOBAL instructions in a first pass and emits them into .rodata before any code.
- Control flow: if/else and loops are lowered to labels and conditional/unconditional jumps. Labels and temps are generated with monotonically increasing counters, guaranteeing uniqueness across the program.
- Dynamic array: the instruction array starts at 1024 entries and doubles on overflow via realloc.

TAC instruction kinds:

- Arithmetic: TAC_ADD, TAC_SUB, TAC_MUL, TAC_DIV, TAC_NEG
- Logical/comparison: TAC_NOT, TAC_AND, TAC_OR, TAC_LT, TAC_GT, TAC_LTE, TAC_GTE, TAC_EQEQ, TAC_NEQ
- Memory: TAC_ASSIGN, TAC_ASSIGN_DEREF, TAC_DEREF, TAC_ADDR
- Control flow: TAC_LABEL, TAC_JUMP, TAC_JUMP_TRUE, TAC_JUMP_FALSE
- Functions: TAC_FUNC_BEGIN, TAC_FUNC_END, TAC_PARAM_DECL, TAC_ARG, TAC_CALL, TAC_RETURN
- Data: TAC_GLOBAL

5.5 codegen_aarch64.c / codegen_aarch64.h

Status	Complete
Input	TACGen* (flat instruction array)
Output	output.s (AArch64 assembly file)

Code generation is the final stage. It walks the TAC instruction array linearly and emits AArch64 assembly to a .s file, which is then assembled and linked by GCC.

Key design decisions:

- Naive stack spilling: every local variable, parameter, and temporary gets its own 8-byte stack slot. No register allocation is performed. Every operation follows load-operate-store: load operands into registers, perform the operation, store the result back to the stack. This is intentionally simple and always correct, even if not optimal.
- Variable table: before emitting any code for a function, codegen scans all TAC instructions between FUNC_BEGIN and FUNC_END and builds a variable table mapping each unique name to a stack offset. Offsets start at 16 to preserve space for the saved x29 and x30 at the bottom of the frame.
- Frame layout: frame size = $\text{align16}(\text{var_count} * 8 + 16)$. The +16 accounts for the saved frame pointer (x29) and link register (x30). All frames are 16-byte aligned as required by the AArch64 calling convention.
- Function prologue/epilogue: the prologue emits `stp x29, x30, [sp, #-frame_size]!` and spills parameters from registers w0-w7 to their stack slots. The epilogue emits a named label (e.g., `fib_end:`) followed by `ldp x29, x30, [sp], #frame_size` and `ret`. Return statements branch to this label.
- Label prefixing: all TAC labels are globally numbered (L0, L1, ...). Codegen prefixes them with the current function name (`fib_L0`) to prevent collisions across functions in the assembly output.
- String literal addressing: string labels in .rodata are loaded using the AArch64 two-instruction `adrp/add` pattern for PC-relative addressing, placed into x0 (64-bit) since they are pointer-sized.
- Pointer operations: `emit_addr` computes the address of a stack variable using `add x0, sp, #offset`. `emit_deref` performs a double load: load the pointer into x0, then load through it. `emit_assign_deref` stores a value through a pointer with `str x1, [x0]`.
- Two-pass emission: globals and string literals are emitted in a first pass before .section .text. Function code is emitted in the second pass.

5.6 main.c

Status	Complete
Input	Source file path (argv[1])
Output	output.s via codegen

main.c is the entry point and pipeline driver. It reads the source file into memory, initializes each stage in order, and passes the output of each stage into the next.

Pipeline:

- Read source file into char*
- Initialize lexer with source
- Initialize parser with lexer, call `parse_program()` to get `ASTNode*` root
- Initialize sema, run `collect_functions()` then `sema_node()` on root
- Initialize TACGen, call `tac_node()` on root to get instruction array
- Initialize CodeGen, call `codegen_run()` to emit output.s

6. Challenges and Reflections

One of the most challenging sections of this project for me was code generation. Working directly with assembly, and designing a system that takes a generic representation of instructions into real, hardware-specific instructions was a huge task. Luckily, going with the naïve stack-spilling approach simplified the process a lot and allowed for me to reach my MVP in time, even if it wasn't as optimal as proper register-allocation. Maybe that could be a project for another time.

Debugging the code generation was also really challenging, as most of the time I just got “bus error” and “segmentation fault” as my error message, with little to no detail or indication of what went wrong. I had to be extremely conscious of memory management in this stage. In fact, there were 2 major bugs I encountered:

1. x29/x30 Stack Corruption

When storing local variables onto the stack, I was starting my stack offset at 0 bytes from the stack pointer. What I did not account for is that the prologue instruction `stp x29, x30, [sp, #-frame_size]!` stores the frame pointer and link register at the bottom of the frame, occupying the first 16 bytes. So my variables were being written directly on top of x29 and x30, corrupting them. When the function tried to return, it restored garbage values from the stack and branched to an invalid address, causing a bus error. The fix was simple once I understood it: start all variable offsets at 16 instead of 0, leaving room for the saved registers. This was my first time working with assembly at this level in practice. I had studied it in Computer Architecture and Compiler Construction, but debugging a real register corruption issue was a completely different experience.

2. Mixed Register Sizes for Pointers

At first, all variable loads and stores used w registers (32-bit), which is correct for int and char. When I added pointer support, I ran into a problem: pointers on AArch64 are 64-bit memory addresses and must be stored and loaded using x registers (64-bit). My stack was also using 4-byte slots for every variable, so a pointer stored with `str x0` would spill into the adjacent variable's slot and corrupt it. I decided on a temporary fix to meet my

course's project deadline: switch all stack slots to 8 bytes regardless of type, and switch all loads and stores to use x registers. This resolved both the slot size and register mismatch issues at the same time, at the cost of memory inefficiency. Both int and char variables were allocated 4 extra bytes of memory, which is wasteful. In the future, I plan to pass some new value from TAC generation that indicates how much memory a variable needs, so I can incorporate that into my code generation and optimize memory allocation.

7. Sample Input/Output

7.1 Sample Input Programs

```
[(base) mateo@Mateos-MacBook-Pro nacc % cat tests/factorial.c
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}

int main() {
    int i = 1;
    while (i <= 10) {
        printf("%d! = %d\n", i, factorial(i));
        i++;
    }
    return 0;
}
```

Example of **factorial.c** input program

7.2 Sample Output

```
[(base) mateo@Mateos-MacBook-Pro nacc % ./nacc tests/factorial.c
[NODE_PROGRAM] 'int'
  [NODE_FUNC] 'factorial' (int)
    [NODE_PARAM] 'n' (int)
    [NODE_BLOCK] '{'
      [NODE_IF] 'if'
        [NODE_BINOP] '<='
          [NODE_IDENT] 'n'
          [NODE_INT_LIT] '1'
        [NODE_BLOCK] '{'
          [NODE_RETURN] 'return'
            [NODE_INT_LIT] '1'
        [NODE_RETURN] 'return'
          [NODE_BINOP] '*'
            [NODE_IDENT] 'n'
            [NODE_CALL] 'factorial'
          [NODE_BINOP] '-'
            [NODE_IDENT] 'n'
            [NODE_INT_LIT] '1'
      [NODE_FUNC] 'main' (int)
        [NODE_BLOCK] '{'
          [NODE_DECL] 'i' (int)
          [NODE_INT_LIT] '1'
          [NODE_WHILE] 'while'
            [NODE_BINOP] '<='
              [NODE_IDENT] 'i'
              [NODE_INT_LIT] '10'
            [NODE_BLOCK] '{'
              [NODE_CALL] 'printf'
                [NODE_STRING_LIT] '%d! = %d\n'
                [NODE_IDENT] 'i'
                [NODE_CALL] 'factorial'
                  [NODE_IDENT] 'i'
              [NODE_POSTFIX] '++'
              [NODE_IDENT] 'i'
            [NODE_RETURN] 'return'
              [NODE_INT_LIT] '0'
```

Generated AST from **factorial.c**

```

[(base) mateo@Mateos-MacBook-Pro nacc % ./nacc tests/factorial.c

[factorial]
    param decl n
    t0 = n LTE 1
    if !t0 goto L1
    return 1
    goto L1
L1:
    t1 = n SUB 1
    arg t1
    t2 = call factorial (1 args)
    t3 = n MUL t2
    return t3
[end factorial]

[main]
    i = 1
L2:
    t4 = i LTE 10
    if !t4 goto L3
    global str0 = "%d! = %d\n"
    arg i
    t5 = call factorial (1 args)
    arg str0
    arg i
    arg t5
    t6 = call printf (3 args)
    t7 = i
    t8 = i ADD 1
    i = t8
    goto L2
L3:
    return 0
[end main]

```

Generated TAC from **factorial.c**

```

[(base) mateo@Mateos-MacBook-Pro nacc % ./nacc tests/factorial.c
.section .rodata
str0:
    .string "%d! = %d\n"
.section .text
.global factorial
factorial:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    str x0, [sp, #16]
    ldr x0, [sp, #16]
    mov x1, #1
    cmp x0, x1
    cset x0, le
    str x0, [sp, #24]
    ldr x0, [sp, #24]
    cbz x0, factorial_L1
    mov x0, #1
    b factorial_end
    b factorial_L1
factorial_L1:
    ldr x0, [sp, #16]
    mov x1, #1
    sub x0, x0, x1
    str x0, [sp, #32]
    ldr x0, [sp, #32]
    bl factorial
    str x0, [sp, #40]
    ldr x0, [sp, #16]
    ldr x1, [sp, #40]
    mul x0, x0, x1
    str x0, [sp, #48]
    ldr x0, [sp, #48]
    b factorial_end
factorial_end:
    ldp x29, x30, [sp], #64
    ret

.global main
main:
    stp x29, x30, [sp, #-64]!
    mov x29, sp
    mov x0, #1
    str x0, [sp, #16]
main_L2:
    ldr x0, [sp, #16]
    mov x1, #10
    cmp x0, x1
    cset x0, le
    str x0, [sp, #24]
    ldr x0, [sp, #24]
    cbz x0, main_L3
    ldr x0, [sp, #16]
    bl factorial
    str x0, [sp, #32]
    adrp x0, str0
    add x0, x0, :lo12:str0
    ldr x1, [sp, #16]
    ldr x2, [sp, #32]
    bl printf
    str x0, [sp, #40]
    ldr x0, [sp, #16]
    str x0, [sp, #48]
    ldr x0, [sp, #16]
    mov x1, #1
    add x0, x0, x1
    str x0, [sp, #56]
    ldr x0, [sp, #56]
    str x0, [sp, #16]
    b main_L2
main_L3:
    mov x0, #0
    b main_end
main_end:
    ldp x29, x30, [sp], #64
    ret

```

Generated AArch64 assembly from **factorial.c**

8. Resources

Textbook: Cooper, Keith D., and Linda Torczon. Engineering a Compiler. 3rd ed., Morgan Kaufmann, 2022. Used throughout my CS4713 Compiler Construction course at UTSA and as a reference in this project for lexer/parser/IR design.

Assembly Reference: Quite a helpful guide and introduction to AArch64 assembly.

<https://mariokartwii.com/arm64/index.html>

AI: All code was written by me. I used Claude strictly as a design/brainstorming tool for design decisions and concept explanations.

Dev Log

03/28/2026

- Set up GitHub repo and project file structure
- Research and AI assisted brainstorming on compiler design
- Update design doc according to specifications and direction I want to take project

03/31/2026

- Continued working on the lexer over the past couple of days. Implemented it myself, used AI for guidance only
- Lexer is now working. It's essentially a bunch of glorified if and switch statements, but it was pretty fun actually building one and seeing it work and tokenize my programs
- Moving on to the parser

```

[(base) mateo@Mateos-MacBook-Pro nacc % cat tests/main.c

int main(int argc, char* argv[]) {

    int a = 5;
    int b = 15;

    return 0;
}
[(base) mateo@Mateos-MacBook-Pro nacc % ./nacc tests/main.c
Token { kind: TOK_INT, value: 'int', line: 3 }
Token { kind: TOK_ID, value: 'main', line: 3 }
Token { kind: TOK_LPAREN, value: '(', line: 3 }
Token { kind: TOK_INT, value: 'int', line: 3 }
Token { kind: TOK_ID, value: 'argc', line: 3 }
Token { kind: TOK_COMMA, value: ',', line: 3 }
Token { kind: TOK_CHAR, value: 'char', line: 3 }
Token { kind: TOK_STAR, value: '*', line: 3 }
Token { kind: TOK_ID, value: 'argv', line: 3 }
Token { kind: TOK_LBRACKET, value: '[', line: 3 }
Token { kind: TOK_RBRACKET, value: ']', line: 3 }
Token { kind: TOK_RPAREN, value: ')', line: 3 }
Token { kind: TOK_LBRACE, value: '{', line: 3 }
Token { kind: TOK_INT, value: 'int', line: 5 }
Token { kind: TOK_ID, value: 'a', line: 5 }
Token { kind: TOK_EQUAL, value: '=', line: 5 }
Token { kind: TOK_INT_LIT, value: '5', line: 5 }
Token { kind: TOK_SEMICOLON, value: ';', line: 5 }
Token { kind: TOK_INT, value: 'int', line: 6 }
Token { kind: TOK_ID, value: 'b', line: 6 }
Token { kind: TOK_EQUAL, value: '=', line: 6 }
Token { kind: TOK_INT_LIT, value: '15', line: 6 }
Token { kind: TOK_SEMICOLON, value: ';', line: 6 }
Token { kind: TOK_RETURN, value: 'return', line: 8 }
Token { kind: TOK_INT_LIT, value: '0', line: 8 }
Token { kind: TOK_SEMICOLON, value: ';', line: 8 }
Token { kind: TOK_RBRACE, value: '}', line: 9 }
(base) mateo@Mateos-MacBook-Pro nacc % █

```

Example of tokenization from my lexer.c on 03/31/2026

04/01/2026

- Designing and documenting the parser strategy. Wrote down CFGs on paper
- Outlining the full Code & Compiler Design section, just working through documentation and getting a rough plan for the rest of the compiler

04/02/2026

- Deep dive into parser implementation. Designed with pen and paper first
- 12-hour session at the cafe. Finished most parse functions
- Parser MVP is complete

```

int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}

int main() {
    int i = 0;
    while (i < 10) {
        printf("%d\n", fib(i));
        i++;
    }
    return 0;
}
(base) mateo@Mateos-MacBook-Pro nacc % ./nacc tests/fibonacci.c
[NODE_PROGRAM] 'int'
[NODE_FUNC] 'fib' (int)
[NODE_PARAM] 'n' (int)
[NODE_BLOCK] '{'
[NODE_IF] 'if'
[NODE_BINOP] '<='
[NODE_IDENT] 'n'
[NODE_INT_LIT] '1'
[NODE_BLOCK] '{'
[NODE_RETURN] 'return'
[NODE_IDENT] 'n'
[NODE_RETURN] 'return'
[NODE_BINOP] '+'
[NODE_CALL] 'fib'
[NODE_BINOP] '-'
[NODE_IDENT] 'n'
[NODE_INT_LIT] '1'
[NODE_CALL] 'fib'
[NODE_BINOP] '-'
[NODE_IDENT] 'n'
[NODE_INT_LIT] '2'
[NODE_FUNC] 'main' (int)
[NODE_BLOCK] '{'
[NODE_DECL] 'i' (int)
[NODE_INT_LIT] '0'
[NODE_WHILE] 'while'
[NODE_BINOP] '<'
[NODE_IDENT] 'i'
[NODE_INT_LIT] '10'
[NODE_BLOCK] '{'
[NODE_CALL] 'printf'
[NODE_STRING_LIT] '%d\n'
[NODE_CALL] 'fib'
[NODE_IDENT] 'i'
[NODE_POSTFIX] '++'
[NODE_IDENT] 'i'
[NODE_RETURN] 'return'
[NODE_INT_LIT] '0'
(base) mateo@Mateos-MacBook-Pro nacc % █

```

Example of a generated AST from my parser.c on 04/02/2026

04/03 - 04/11/2026

- Continuing lectures, studying Engineering a Compiler, taking the course midterm, advancing other coursework

04/20/2026 – 04/24/2026

- Started semantic analysis. Set up structure, designed the 2 pass workflow (collect function signatures first, then walk the AST and annotate/validate)
- Added support for void pointers in the parser and comment skipping in the lexer
- Implemented
- Completed sema functionality: type checking, declaration checking, and annotation. Sema MVP is complete.

04/25/2026 – 05/06/2026

- Began on TAC generation. Set up structs, recursive walks through AST, print/debug statements. Ready to begin implementing
- Progressed through TAC generation relatively smoothly, simple operations, but had to spend a lot of time to fine tune and get right, because codegen depends on it.
- TAC generation is complete, ready for codegen.

05/06/2026 – 05/09/2026

- 3 full days of extreme grind. Working through code generation. Started like usual by setting up structs, codegen emit methods, walking functionality, etc.
- Running into difficult corruption bugs, bus errors and segmentation faults, no known cause, will continue debugging
- Found and fixed bugs caused by memory mismanagement
- Basic codegen is working, went with naïve stack spilling approach for MVP
- Codegen MVP is complete.

05/09/2026

- Compiler MVP complete. Finished sema, TAC gen, and AArch64 codegen.
- Tested on Raspberry Pi 4B running Ubuntu Desktop (AArch64). fibonacci, factorial, pointer programs all produce correct output

05/10/2026

- Implemented support for int and char arrays, had to pull an all nighter to squeeze this into my final project submission. Style points!
- Finished last feature of Compiler. **Project is complete!**


```

40 //calls down the precedence chain
39 ASTNode* left = parse_or(parser);
38
37 if(!check(parser, TOK_EQUAL) && !check(parser, TOK_PLUSEQ) && !check(parser, TOK_MINUSEQ) && !check(parser, TOK_MULTSEQ) && !check(parser, TOK_DIVEQ)) {
36     return left;
35 }
34
33 // consume whichever assignment operator it is
32 match(parser, TOK_EQUAL) || match(parser, TOK_PLUSEQ) ||
31 match(parser, TOK_MINUSEQ) || match(parser, TOK_MULTSEQ) ||
30 match(parser, TOK_DIVEQ);
29
28 Token operator = parser->previous_token;
27 ASTNode* right = parse_assign(parser); //recursive call is right-associative, which is what we want for assignment
26
25 ASTNode* node = make_node(NODE_ASSIGN, operator); //must store previous token on assignment
24 node->left = left;
23 node->right = right;
22 return node;
21 }
20
19 //entry point into chain of precedence for things like assignment, logical comparison, arithmetic, unary, primary, etc.
18 ASTNode* parse_expr(Parser* parser) {
17     return parse_assign(parser);
16 }
15
14 ASTNode* parse_stmt(Parser* parser) {
13     ASTNode* node = parse_expr(parser);
12     expect(parser, TOK_SEMICOLON, "expected ';' after expression");
11     return node;
10 }
9
8 ASTNode* parse_decl(Parser* parser) {
7     TypeKind type = consume_type(parser);
6
5     Token token = expect(parser, TOK_ID, "expected variable name");
4
3     if(type == TYPE_VOID) { error(token.line, "cannot have a variable of type void"); }
2
1 //check for array declaration
344 if()
1
2 //initialization
3 ASTNode* left = NULL;
4 //if this declaration is also initializing
5 if(match(parser, TOK_EQUAL)) {
6     left = parse_expr(parser);
7 }
8
9 expect(parser, TOK_SEMICOLON, "expected ';' after declaration");
10
11 ASTNode* node = make_node(NODE_DECL, token);
12 node->type = type;
13 node->token = token;
14 node->left = left;
15
16 return node;
17 }
18
19 ASTNode* parse_return(Parser* parser) {
20     Token token = expect(parser, TOK_RETURN, "expected return statement");
21
22     ASTNode* left = NULL;
23     if(!check(parser, TOK_SEMICOLON)) {
24         left = parse_expr(parser);
25     }
26
27 return node;
28 }
29
30 //...
31
32 src/parser.c [+]
33
34 344,8
35 47%

```

Implementing support for arrays 05/09/2026